

Dokumentacja Projektowa JAVA

System Wizualizacji Grafów

Mateusz Bombola

Kajetan Karwacki

26 maja 2026

Spis treści

1	Wstęp i Cel Projektu	3
2	Specyfikacja Funkcjonalna	3
2.1	Realizowane Funkcje	3
2.2	Interfejs Użytkownika (GUI)	3
3	Architektura i Implementacja	4
3.1	Podział na Moduły	4
3.2	Struktury Danych	4
4	Kluczowe Mechanizmy i Integracja	4
4.1	Współpraca z silnikiem C	4
4.2	Dynamiczne Skalowanie Współrzędnych	4
4.3	Automatyczne Centrowanie	5
5	Obsługa Plików i Błędy	5
5.1	Reakcje na błędy (Kody wyjątkowe)	5
6	Dokumentacja Graficzna Interfejsu	6

1 Wstęp i Cel Projektu

Celem projektu jest okienkowa aplikacja w języku Java (wykorzystująca bibliotekę Swing), służąca do interaktywnej wizualizacji topologii grafów planarnych na płaszczyźnie 2D. Program pełni rolę graficznego interfejsu użytkownika (GUI) oraz orkiestratora, który integruje się z zewnętrznym silnikiem obliczeniowym w języku C. Aplikacja pobiera zdefiniowane krawędzie, wywołuje program w C w celu uzyskania zoptymalizowanych współrzędnych węzłów, a następnie płynnie renderuje i pozwala na modyfikację wygenerowanego układu.

2 Specyfikacja Funkcjonalna

2.1 Realizowane Funkcje

- Wczytywanie struktury grafu z plików tekstowych (.txt).
- Interaktywne renderowanie układu z dynamicznym kolorowaniem krawędzi (gradient od zieleni do czerwieni) na podstawie wczytanych wag.
- Wybór algorytmu obliczeniowego (Fruchterman-Reingold lub Tutte Embedding) z poziomu interfejsu graficznego.
- Swobodne manipulowanie kamerą: przybliżanie i oddalanie (zoom) za pomocą kółka myszy oraz przesuwanie widoku (panning).
- Edycja układu w czasie rzeczywistym poprzez ręczne przeciąganie wierzchołków metodą „Drag & Drop” oraz precyzyjną edycję w panelu właściwości.
- Zapis wyliczonych współrzędnych do pliku .txt oraz eksport widoku obszaru roboczego do pliku graficznego (.png).

2.2 Interfejs Użytkownika (GUI)

Program działa w trybie okienkowym. Składa się z górnego paska narzędziowego (Menu) obsługującego wejście/wyjście plików, centralnego płótna renderującego (`GraphPanel`) oraz prawego panelu bocznego, który skupia kontrolki sterujące silnikiem obliczeniowym, pola edycyjne zaznaczonego węzła i opcje wyświetlania (włączanie/wyłączanie etykiet oraz wag).

3 Architektura i Implementacja

3.1 Podział na Moduły

Aplikacja opiera się na podejściu obiektowym i dzieli się na następujące klasy:

- **GraphApp**: Klasa główna odpowiadająca za konstrukcję głównego okna (**JFrame**), układ komponentów, walidację interakcji oraz zarządzanie zintegrowanym procesem C.
- **GraphPanel**: Silnik renderujący (**JPanel**). Nadpisuje metodę `paintComponent` w celu rysowania grafiki wektorowej za pomocą **Graphics2D** oraz obsługuje nasłuchiwanie zdarzeń myszy.

3.2 Struktury Danych

Dane topologiczne przechowywane są przy pomocy lekkich klas:

- **Node**: Przechowuje identyfikator (`id`) oraz pozycje `x` i `y` (zmiennoprzecinkowe) dla każdego węzła. Węzły mapowane są w słowniku `Map<Integer, Node>`.
- **Edge**: Definiuje połączenie między dwoma wierzchołkami (`u`, `v`) wraz z przypisaną mu wagą (`weight`). Krawędzie przechowywane są w kolekcji `List<Edge>`.

4 Kluczowe Mechanizmy i Integracja

4.1 Współpraca z silnikiem C

Program wykorzystuje obiekt `ProcessBuilder` do natywnego uruchomienia wygenerowanego pliku `graph_layout.exe`. Dla zapewnienia bezproblemowej pracy w strukturze projektu, aplikacja używa dynamicznego radaru ścieżek – przeszukuje predefiniowaną listę folderów (m.in. `Algorytm c/pliki_zrodlowe_2_grupy/`). Komenda uruchomienia budowana jest z uwzględnieniem specyfiki drugiej grupy (przekazywanie ścieżki wejściowej bez flagi oraz ewentualne użycie flagi `-a` dla algorytmu Tutte’a).

4.2 Dynamiczne Skalowanie Współrzędnych

Algorytmy dystansowe mogą zwracać wartości znormalizowane do ułamków dziesiętnych. Przed rzutowaniem pozycji na piksele (`int`), Java analizuje przestrzeń (`getDynamicScale()`). Gdy najwyższa wartość współrzędnej jest mniejsza od zdefiniowanego progu (15 jednostek), aplikacja stosuje do rysowania globalny mnożnik 150.0. Eliminuje to błąd numeryczny objawiający się „zapadnięciem” grafu.

4.3 Automatyczne Centrowanie

Metoda `resetView()` iteruje po załadowanych węzłach w celu odnalezienia wartości `minX`, `maxX`, `minY` oraz `maxY`. Z uzyskanej obwiedni wyliczany jest środek ciężkości grafu, a wartości przesunięcia kamery (`px`, `py`) są odpowiednio korygowane. Dzięki temu układ po obliczeniach zawsze wyświetlany jest dokładnie pośrodku ekranu użytkownika, niezależnie od bazowych wartości wynikowych.

5 Obsługa Plików i Błędy

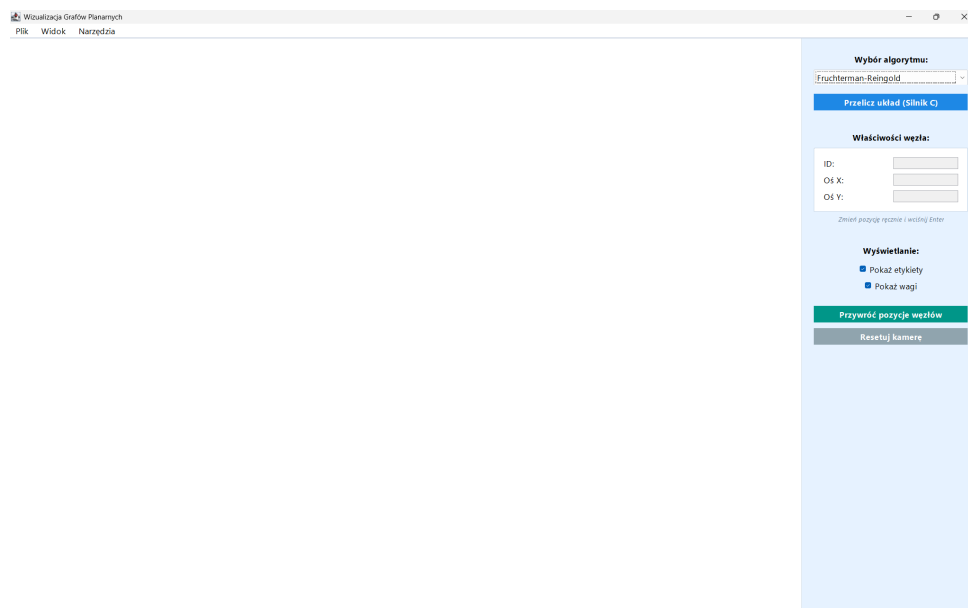
Moduł I/O obsługuje ładowanie i zapis plików tekstowych, odczytując linie i weryfikując poprawność długości wprowadzonych danych.

5.1 Reakcje na błędy (Kody wyjątkowe)

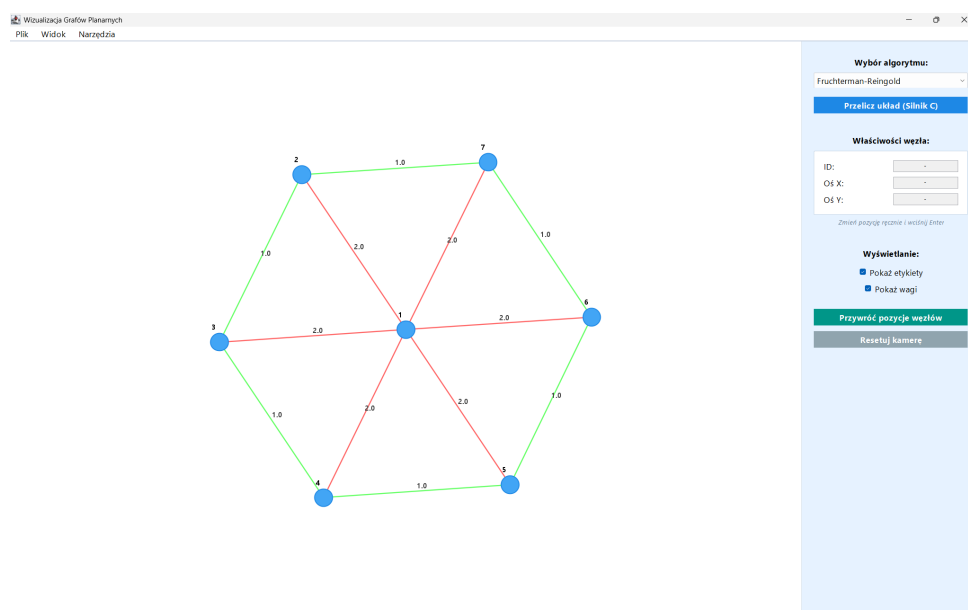
Aplikacja zabezpiecza działanie systemu i informuje użytkownika poprzez wyskakujące okna (`JOptionPane`):

Sytuacja Wyjątkowa	Reakcja Systemu Java
Brak danych krawędzi	Blokada wywołania silnika C i komunikat „Najpierw wczytaj strukturę krawędzi!”.
Nieznalezienie pliku EXE	Włączenie okna <code>JFileChooser</code> pozwalającego użytkownikowi wskazać ręcznie lokalizację silnika.
Błąd w module C	Przechwycenie kodu powrotu oraz wiadomości ze strumienia błędów (<code>ErrorStream</code>) i wyświetlenie alertu „Awaria w C”.

6 Dokumentacja Graficzna Interfejsu



Rysunek 1: Główne okno programu przed załadowaniem struktury grafu



Rysunek 2: Wycentrowany układ wierzchołków uzyskany po wykonaniu obliczeń przez silnik C